01. THÔNG TIN NHÓM VÀ MÔ HÌNH

BÁO CÁO GIỮA KỲ — XẾP HÌNH 8 Ô NÂNG CAO

Học phần: Biểu diễn tri thức và tìm kiếm nâng cao
Giảng viên: TS. Nguyễn Bích Vân — IAI–UET–VNU

Nhóm / lớp: Nhóm 1 / lớp AI2014_1 **Ngày giao / ngày nộp:** 11/10/2026 – 24/10/2026
Môi trường Python / hệ điều hành: Python 3.12.10 / Windows 11

Thành viên / MSSV	Phần chủ trì	2 test phụ trách	Kiểm tra chéo
1. Đoàn Mai Trúc Linh / 25022911	du_lieu_nhom.json, tao_du_lieu.py, dong_goi.py, README_NHOM1.md, DongGop.csv, BaoCao.pdf	test_trang_thai_dau_trung_dich & test_chuyen_ dich_ mot_ buoc_ngang_doc	BaoCao.pdf và TinhTay.pdf
2. Trần Thùy Dương / 25022816	BaoCao.pdf, puzzle.py, search.py	test_dich_tuy_chinh & test_heuristic_bang_0_tai_dich_va_bo_qua_o_trong	du_lieu_nhom.json và tu_tao.csv
3. Hoàng Thị Ngọc Sơn / 25023009	TinhTay.pdf, BaoCao.pdf	test_vo_nghiem_do_parity & test_du_lieu_sai_va_hanh_dong_vuot_bien	test_nhom.py
4. Nguyễn Thanh Bình / 25022796	ThuyetTrinh.pdf, Timeline-presentation.md, test_nhom.py, output/ket_qua_test.txt	test_doi_chieu_astar_zero_va_ucs & test_doc_lap_trace_va_gioi_han_tai_nguyen	README_NHOM1.md
5. Phạm Hồ Minh Thư / 25023057	BaoCao.pdf, output/bat_buoc.csv và output/tu_tao.csv	test_duong_di_hop_le_va_dau_vao_bat_bien & test_bfs_khong_toi_uu_giao_directional	ThuyetTrinh.pdf

1. Bài toán nhóm đang giải

Trạng thái, hành động, điều kiện đích, hàm chuyển, giải thích vì sao chỉ vị trí ô trống là chưa đủ:

- Trạng thái: Là một hoán vị của bộ 9 số (0, 1, 2, 3, 4, 5, 6, 7, 8) đại diện cho cấu hình các ô trên bảng 3x3, trong đó số 0 tượng trưng cho ô trống. Trong mã nguồn, mỗi trạng thái được biểu diễn bằng 1 tuple 9 phần tử đọc theo hàng từ trái sang phải.

- Hành động: Đổi 0 (ô trống) với ô kề trên/dưới/trái/phải, với điều kiện di chuyển ô 0 theo hướng đó không vượt ra ngoài ranh giới bảng 3x3.

- Điều kiện đích: Là cấu hình bảng mục tiêu cần đạt được, mặc định G = (1, 2, 3, 4, 5, 6, 7, 8, 0). Bản của nhóm có hỗ trợ dynamic goal được truyền vào. Đích đạt được khi current state == goal.

- Hàm chuyển trạng thái: Nhận vào trạng thái hiện tại và hành động hợp lệ, trả về trạng thái mới cùng chi phí hành động. Là hàm neighbors(state, model, order) trong tệp puzzle.py được cung cấp.

- Không được dùng duy nhất vị trí 0 làm trạng thái vì một biểu diễn trạng thái hợp lệ trong bài toán tìm kiếm phải chứa đủ thông tin để xác định cấu hình hiện tại, kiểm tra đích và sinh các trạng thái kế tiếp. Nếu chỉ lưu vị trí của ô 0, không gian trạng thái chỉ có 9 giá trị (0, 1, ... , 8), bỏ qua (gộp làm một) rất nhiều trạng thái của bài toán; ví dụ, 2 bảng khác nhau hoàn toàn như (0, 1, 2, 3, 4, 5, 6, 7, 8) và (0, 8, 7, 6, 5, 4, 3, 2, 1) sẽ bị coi là cùng 1 trạng thái.

Mô hình	Chi phí một bước	Mục tiêu tối ưu
unit	U/D/L/R đều bằng 1	Ít bước nhất = ít chi phí nhất
directional	U/D bằng 1; L/R bằng 2	Ít tổng chi phí nhất, không chỉ ít bước nhất

Nguồn mã đã dùng nguyên / các tệp nhóm thay hoặc tự viết:

Các nguồn mã đã dùng nguyên: puzzle.py, run.py, tao_du_lieu.py, search.py, dong_goi.py, kiem_tra_moi_truong.py, data_case.py bao gồm thuật toán BFS, UCS, A*, cơ chế sinh con neighbors, kiểm tra is_solvable, validate_state và khởi tạo make_heuristic.

Các tệp nhóm tự viết gồm: TinhTay.pdf, ThuyetTrinh.pdf, README_NHOM1.md, test_nhom.py, du_lieu_nhom.json, Dong_Gop.csv, ket_qua_test.txt, BaoCao.pdf:

- TinhTay.pdf: Tính tay h_1, h_2, h_3 tại H=(1,3,6,5,0,2,4,7,8). Thực hiện 6 lần lấy ra hợp lệ của A* với h_2, lập bảng trạng thái OPEN. Chứng minh đường đi từ H về G là tối ưu. Giải thích 3 lỗi lập luận (đếm cả ô 0 trên bàn V, giữ tọa độ cũ khi đổi goal, phân biệt parity và limit).
- ThuyetTrinh.pdf: Giới thiệu và trình bày kết quả thực hành làm việc nhóm.
- README_NHOM1.md: Trình bày môi trường đã sử dụng, các file dữ liệu đầu ra, dữ liệu riêng nhóm sử dụng và hạn chế về tài nguyên.
- test_nhom.py: 10 test do nhóm thiết kế theo yêu cầu.
- du_lieu_nhom.json: dữ liệu nhóm sử dụng với Goal khác và N1, N2 mới từ 16 bước sắp xếp.
- Dong_Gop.csv: Ghi nhiệm vụ/hàm test của từng thành viên.
- ket_qua_test.txt: ghi log kết quả sau khi chạy test nhóm.
- BaoCao.pdf: Báo cáo kết quả làm việc của từng thành viên nhóm.

02. TRÌNH BÀY THUẬT TOÁN VÀ HEURISTIC

2. A* trong chương trình của nhóm

Giải thích f, g, h bằng lời của nhóm:

- f(s) = g(s) + h(s) là tổng chi phí ước lượng cho toàn bộ đường đi từ trạng thái bắt đầu qua s đến trạng thái đích, được dùng làm tiêu chí xếp thứ tự ưu tiên cho các trạng thái trong biên (ưu tiên chọn trạng thái có f nhỏ nhất). Trong trường hợp hòa f, áp dụng quy tắc phá hòa FIFO (trạng thái sinh ra trước được lấy trước) bằng biến đếm số thứ tự serial.

- g(s) là tổng chi phí thực tế từ trạng thái bắt đầu đến trạng thái hiện tại.

- h(s) là chi phí ước lượng (không vượt quá chi phí thực tế) để đi từ trạng thái hiện tại tới đích. Trong bài toán này, h(s) được tính bằng h_1(s) - số ô sai vị trí, h_2(s) - tổng khoảng cách Manhattan, bỏ qua ô 0 (đối với mô hình unit) hoặc h_3(s) - khoảng cách Manhattan có trọng số, bỏ qua ô 0 (đối với mô hình directional) dựa trên trạng thái đích được truyền vào.

Khác biệt BFS/UCS/A*:

- BFS: Biên dùng hàng đợi. Thuật toán tìm ra lời giải có ít bước đi nhất (chiều dài đường lời giải ngắn nhất), không quan tâm đến chi phí của mỗi bước đi. Với mô hình unit của bài toán, do mỗi lần chuyển trạng thái đều có chi phí bằng 1 nên BFS vẫn tìm được lời giải tối ưu, nhưng thời gian tìm kiếm sẽ lâu hơn A* do BFS mở rộng loang ra theo từng tầng. Tuy nhiên, với mô hình directional (chi phí dịch chuyển sang trái/phải bằng 2), BFS không đảm bảo tối ưu do lúc này số bước ít nhất chưa chắc đồng nghĩa với chi phí nhỏ nhất.

- UCS: Biên dùng hàng đợi ưu tiên theo giá trị g(s) (tương đương A* với h(s) = 0). Thuật toán đảm bảo tìm được lời giải tối ưu ở mọi mô hình nhưng phải mở rộng nhiều trạng thái. Với mô hình unit, BFS và UCS cho lời giải và số trạng thái mở rộng giống nhau, nhưng BFS nhanh hơn nhờ dùng hàng đợi (thêm/lấy phần tử cần O(1)) thay vì dùng hàng đợi ưu tiên như UCS (cần O(logN) để thêm/lấy phần tử).

- A*: Biên dùng hàng đợi ưu tiên theo giá trị f(s). Thuật toán kết hợp chi phí thực tế g(s) với ước lượng h(s) để vừa đảm bảo tìm được lời giải tối ưu ở mọi mô hình, vừa giảm tối đa số trạng thái phải mở rộng so với UCS.

Bước thuật toán	Mã giả do nhóm viết / giải thích	Tệp và dòng tương ứng

Khởi tạo	h = make_heuristic()
	serial = count() # Biến đếm để phá hòa FIFO
	pq = [(h[start], next(serial), 0, start)]
	best_g = {start: 0}
	parent = {start: None}
		search.py dòng 42: tạo heuristic, hàm make_heuristic() chi tiết ở dòng 62-85 trong puzzle.py;
		dòng 52–59: tạo nút bắt đầu, best_g, bộ đếm serial và heap, đưa nút bắt đầu vào heap.

Lấy nút / xử lý hòa	while pq not empty: f, _, g, u = heappop(pq)	search.py dòng 87–93: lấy nút khỏi biên; dòng 59 và 140: dùng serial để giữ thứ tự FIFO khi các nút hòa f.
Bỏ bản ghi cũ	if g != best_g[u]: continue	search.py dòng 95–99: kiểm tra nút đã lỗi thời theo best_g/expanded_g và bỏ qua khi pop.
Kiểm tra đích	if u == goal: return path, path_cost	search.py dòng 106–113: kiểm tra đích sau khi lấy ra nút hợp lệ rồi gọi finish.

Bước thuật toán	Mã giả do nhóm viết / giải thích	Tệp và dòng tương ứng

Sinh con / cập nhật giá	for action, v, cost in neighbors(u):
	newg = g + cost
	if new_g < best_g.get(v, inf):
	best_g[v] = new_g
	parent[v] = u
	push (h[v] + new_g, next(serial), new_g, v) into pq
		search.py dòng 125–141: duyệt các con, tính new_g, chỉ cập nhật khi tốt hơn và đưa nút mới vào heap. Hàm sinh trạng thái và tính chi phí cạnh là neighbors() trong puzzle.py ở dòng 42–59.

Lưu cha / dựng đường	- Lưu cha: là dòng parent[v] = u ở bước trên. - Dựng đường (thực hiện khi u == goal): path = [] curr = u while curr is not None: append current to path curr = parent[curr] reverse path # for correct order from start -> goal return path, path_cost	search.py dòng 17–22: tạo cấu trúc Node lưu parent/ action, dòng 136 gắn cha khi thêm nút con; dòng 63–76: finish() lần ngược cha và đảo đường đi về đúng thứ tự.

3. Vì sao heuristic dùng được?

Hàm	Định nghĩa / bỏ ô 0 thế nào?	Điều kiện theo mô hình
h1	- Định nghĩa: Hàm ước lượng dựa trên số ô số sai vị trí. - Cách xử lý ô 0: Không tính vào số ô sai vị trí. - Giải thích: Ô 0 đại diện cho ô trống, chỉ di chuyển vì có 1 ô số khác di chuyển. Nếu đếm cả ô 0 vào h1 thì sẽ bị lặp chi phí của 1 ô số khác, dẫn đến khả năng h1(s) có thể đánh giá cao hơn chi phí thực tế khiến hàm ước lượng này không chấp nhận được.	- Dùng cho mô hình unit và có thể là cận dưới chấp nhận được của directional.
h2	- Định nghĩa: Hàm ước lượng dựa trên tổng sai số theo Manhattan của các ô số (so với vị trí đúng trong trạng thái đích được truyền vào). - Cách xử lý ô 0: Không tính vào số ô sai vị trí. - Giải thích: Tương tự h_1 (nếu tính cả ô 0 thì bị lặp sai số Manhattan).	- Dùng cho mô hình unit và có thể dùng làm cận dưới chấp nhận được cho directional.
h3	- Định nghĩa: Tương tự h2 nhưng là Manhattan có trọng số cho khoảng cách theo phương ngang ($\|c_i - c_i^*\|$). - Cách xử lý ô 0 & Giải thích: Tương tự h2.	- Dùng cho mô hình directional.

Lập luận h1 ≤ h2 và cận dưới cho từng mô hình:

1) Với mô hình unit:

* Chứng minh $h_1(s) \le h_2(s)$:
Với mỗi ô i trong {1, 2, 3, 4, 5, 6, 7, 8}, nếu ô này đã ở vị trí đúng trong trạng thái đích thì khoảng cách Manhattan $|r_i - r_i^*| + |c_i - c_i^*| = 0$, nếu lệch so với vị trí đúng thì khoảng cách Manhattan $|r_i - r_i^*| + |c_i - c_i^*| \ge 1$. Lấy tổng sai số của tất cả 8 ô số:

$h_2(s)$ = tổng khoảng cách Manhattan của 8 ô số = tổng khoảng cách Manhattan của các ô số sai vị trí ≥ số ô số sai vị trí = $h_1(s)$

* Chứng minh $h_2(s) \le h^*(s)$:
Giả sử bỏ qua vật cản giữa các ô số và bỏ qua hạn chế phải di chuyển qua ô 0 (mỗi ô số có thể tự do di chuyển độc lập sang ô kề). Để đưa ô i ở vị trí (r_i, c_i) về vị trí đích (r_i^*, c_i^*), bài toán nới lỏng trên cần ít nhất $|r_i - r_i^*| + |c_i - c_i^*|$

bước đi. Mỗi lần di chuyển một ô số bất kỳ chỉ có thể làm giảm tổng khoảng cách Manhattan của toàn bộ bàn cờ đi tối đa là 1. Do đó, để tổng khoảng cách Manhattan từ $h_2(s)$ giảm về 0 (đạt trạng thái đích) cần ít nhất $h_2(s)$ bước đi. Mà mọi lời giải hợp lệ của bài toán 8-puzzle gốc cũng là một lời giải hợp lệ cho bài toán nới lỏng. Vậy số bước đi thực tế h*(s) không thể nhỏ hơn số bước trong bài toán nới lỏng:

$$h_2(s) \le h^*(s)$$

Kết luận: Trong mô hình unit, $h_1(s) \le h_2(s) \le h^*(s)$ với mọi trạng thái s. Cận dưới của unit là $h_1(s)$ và $h_2(s)$.

2) Với mô hình directional:

* $h_1(s) \le h_2(s)$ theo chứng minh ở phần trước.

* Chứng minh $h_2(s) \le h_3(s)$:
Ta có $|r_i - r_i^*| + 2|c_i - c_i^*| = (|r_i - r_i^*| + |c_i - c_i^*|) + |c_i - c_i^*| \ge |r_i - r_i^*| + |c_i - c_i^*|$. Lấy tổng trên tất cả 8 quân cờ:

$$h_3(s) = \text{tổng từ } i = 1 \text{ đến } i = 8 \text{ của } |r_i - r_i^*| + 2|c_i - c_i^*|$$
$$\ge \text{tổng từ } i = 1 \text{ đến } i = 8 \text{ của } |r_i - r_i^*| + |c_i - c_i^*| = h_2(s)$$

* Chứng minh $h_3(s) \le h^*(s)$:
Giả sử bỏ qua vật cản giữa các ô số và bỏ qua hạn chế phải di chuyển qua ô 0 (mỗi ô số có thể tự do di chuyển độc lập sang ô kề). Để đưa ô i ở vị trí (r_i, c_i) về vị trí đích (r_i^*, c_i^*), bài toán nới lỏng trên cần ít nhất $|r_i - r_i^*| + |c_i - c_i^*|$ bước đi. Mỗi lần di chuyển một ô số bất kỳ (dọc tốn chi phí 1, ngang tốn chi phí 2) chỉ có thể làm giảm tổng khoảng cách Manhattan có trọng số của toàn bàn cờ đi một lượng tối đa bằng chính chi phí của bước đi đó (giảm tối đa 1 cho bước dọc và 2 cho bước ngang). Do đó, để tổng khoảng cách Manhattan có trọng số từ $h_3(s)$ giảm về 0 (đạt trạng thái đích) cần ít nhất $h_3(s)$ bước đi. Mà mọi lời giải hợp lệ của bài toán 8-puzzle gốc cũng là một lời giải hợp lệ cho bài toán nới lỏng. Vậy số bước đi thực tế h*(s) không thể nhỏ hơn số bước trong bài toán nới lỏng:

$$h_3(s) \le h^*(s)$$

Kết luận: Trong mô hình directional, $h_1(s) \le h_2(s) \le h_3(s) \le h^*(s)$ với mọi trạng thái s. Các cận dưới của h*(s) là $h_1(s)$, $h_2(s)$ và $h_3(s)$.

Lập luận nhất quán trên đúng một cạnh:

Xét bước chuyển hợp lệ từ trạng thái s sang trạng thái s' qua cạnh e = (s, action, s') có chi phí c(s, a, s'):

- **h_1:**
  - Mỗi bước di chuyển 1 ô số nên số ô sai vị trí thay đổi tối đa 1 ô (tăng 1/giảm 1/giữ nguyên)
  => $0 \le h_1(s) - h_1(s') \le 1$.
  - Trong mô hình unit, c = 1. Với mô hình directional, c = 1 hoặc c = 2. Suy ra $c \ge 1$.
  Vậy ta luôn có $h_1(s) - h_1(s') \le 1 \le c(s, a, s')$, tương đương $h_1(s) \le c(s, a, s') + h_1(s')$.
  Do đó **h_1 nhất quán**.
- **h_2:**

  - Mỗi bước di chuyển 1 ô số khiến khoảng cách Manhattan của ô đó tăng 1 hoặc giảm 1.
  => $|h_2(s) - h_2(s')| = 1 \Rightarrow h_2(s) - h_2(s') \le 1$.
  - Trong mô hình unit, c = 1. Với directional, c = 1 hoặc c = 2. Suy ra $c \ge 1$.

  Vậy luôn có $h_2(s) - h_2(s') \le 1 \le c(s, a, s')$, tương đương $h_2(s) \le c(s, a, s') + h_2(s')$.
  Do đó **h_2 nhất quán**.

- **h_3:**
  - Nếu di chuyển ô theo chiều dọc (U/D): c = 1, $h_3(s) - h_3(s') \le 1 = c$.

- Nếu di chuyển ô theo chiều ngang (L/R): c = 2 (khoảng cách theo phương ngang đổi 1 nhưng được nhân với trọng số 2), $h_3(s) - h_3(s') \leq 2 = c$.
Vậy luôn có $h_3(s) - h_3(s') \leq c$, tương đương $h_3(s) \leq c(s, a, s') + h_3(s')$.
Do đó **h_3 nhất quán**.

Điều kiện A* tối ưu; vì sao không dừng khi vừa sinh đích:

1) Để đảm bảo A* tìm được lời giải tối ưu:

- Heuristic h(s) chấp nhận được ($h(s) \leq h^*(s)$) và thỏa mãn tính nhất quán ($h(s) \leq h^*(s) + c(s, a, s')$ với mọi cạnh chuyển trạng thái e = (s, a, s').

- Chi phí bước chuyển trên mọi cạnh e dương: $c(s, a, s') > 0$.

- A* chỉ dừng và công nhận lời giải khi trạng thái đích được lấy ra khỏi biên tìm kiếm, không dừng khi vừa sinh đích (khi mới heappush vào priority queue).

2) Không dừng khi vừa sinh đích vì:

- Việc trạng thái đích được thêm vào hàng đợi (sinh ra) chỉ báo hiệu một đường đi hợp lệ đến G vừa được tìm thấy, chưa chắc đó là đường tối ưu.

- Giả sử tồn tại trạng thái u còn trong OPEN trên một đường khác tới đích. Vì h chấp nhận được nên $f(u) = g(u) + h(u) \leq cost^*(u) = g(u) + h^*(u)$, với $cost^*(u)$ là chi phí thực tế của đường từ S đi qua u đến G. Nếu $f(u) < g_{đích}$ thì có thể còn đường qua u rẻ hơn => dừng ngay sẽ bỏ lỡ nghiệm tối ưu.

- Ngược lại, khi đích được lấy khỏi biên thì $f(goal) = g_{đích}$ là giá trị nhỏ nhất trong biên, tức $g_{đích} \leq f(u) \leq cost^*(u)$ với mọi trạng thái u còn lại. Lúc này không còn đường nào có chi phí thực tế thấp hơn nên dừng thuật toán là đảm bảo tối ưu.

03. TÍNH TAY VÀ ĐỐI CHIẾU VẾT CHẠY

4. Trạng thái H và dự đoán trước khi chạy

H = (1, 3, 6, 5, 0, 2, 4, 7, 8). PDF tính tay chi tiết: TinhTay.pdf, trang 1,2,3,4.
Dự đoán được ghi vào ngày / thời điểm: 23/10/2026 – 19h00.

h1(H)	h2(H)	h3(H)	Ghi chú đóng góp quân
7	8	13	Quân 1 đúng chỗ (0). Quân 2 lệch cả hàng và cột nên đóng góp nhiều nhất ($h_2 = 2$, $h_3 = 3$). Quân 3, 5, 7, 8 chỉ lệch cột ($h_2 = 1$, $h_3 = 2$). Quân 4, 6 chỉ lệch hàng ($h_2 = 1$, $h_3 = 1$). Ô trống không tính.

Lần	Nhãn / trạng thái lấy ra	g	h	f	Khớp tay–code?
1	S = (1,3,6,5,0,2,4,7,8)	0	8	8	Khớp
2	L = (1,3,6,0,5,2,4,7,8)	1	7	8	Khớp
3	R = (1,3,6,5,2,0,4,7,8)	1	7	8	Khớp
4	LD = (1,3,6,4,5,2,0,7,8)	2	6	8	Khớp
5	RU = (1,3,0,5,2,6,4,7,8)	2	6	8	Khớp
6	LDR = (1,3,6,4,5,2,7,0,8)	3	5	8	Khớp

OPEN sau một bước nhóm chọn; nơi FIFO quyết định thứ tự: Trang 3 TinhTay.pdf / tệp trace: tinh_tay_HAND_unit_manhattan_trace.csv .

Mô hình	Dãy hành động / đường	Số bước	Cost	Lý do tối ưu
unit	R, U, L, D, L, D, R, R	8	8	$h_2(H) = 8$ là cận dưới của chi phí thật và bằng chi phí đường tìm được nên đường tối ưu.
directional	R, U, L, D, L, D, R, R	8	13	$h_3(H) = 13$ là cận dưới và bằng chi phí đường tìm được (5 bước ngang × 2 + 3 bước dọc × 1) nên đường tối ưu.

5. Các phản ví dụ đã phát hiện

Đếm cả 0 trên V:

- h_sai(V) = 2 (ô (1,2) có 0 thay vì 6, ô (2,2) có 6 thay vì 0).
- Chi phí thật h*(V) = 1 (một bước D, tốn 1 ở cả unit và directional).
- Cạnh vi phạm là V → G (chi phí c = 1): h_sai(V) = 2 > c + h_sai(G) = 1 + 0 = 1, nên không nhất quán và cũng không chấp nhận được (2 > 1).

Đổi goal nhưng giữ tọa độ cũ:
Với G' = (1,2,3,4,5,6,0,7,8) nhưng vẫn dùng tọa độ đích của G, hàm sai cho h(G') = 2 (quân 7 lệch 1, quân 8 lệch 1) thay vì 0. Điều kiện bị vi phạm là h(đích) = 0, nên không chấp nhận được. Cách sửa: tính tọa độ đích từ goal đang truyền vào.

Parity và limit khác nhau:
Z = (1, 2, 3, 4, 5, 6, 8, 7, 0) có 1 nghịch thế (lẻ), G có 0 (chẵn); parity bất biến với bàn 3×3 nên Z chắc chắn vô nghiệm, kết luận bằng chứng minh mà không cần tìm kiếm. Còn `limit` chỉ là thuật toán hết tài nguyên trước khi tìm ra đường, nên chưa biết có nghiệm hay không và không được ghi là vô nghiệm.

Điểm dự đoán sai hoặc hiểu rõ hơn sau đối chiếu:

- **Dự đoán đúng:** Sáu lần lấy ra đầu của A* (unit, h_2, FIFO) đúng thứ tự S → S/L → S/R → S/L/D → S/R/U → S/L/D/R, khớp với trace tinh_tay_HAND_unit_manhattan_trace.csv. Các giá trị g = 0, 1, 1, 2, 2, 3 và h_2 = 8, 7, 7, 6, 6, 5 cũng khớp hoàn toàn. Cost tối ưu 8 (unit) và 13 (directional) được chứng minh bằng tay ở TinhTay.pdf (B3), nơi cận dưới $h_2(H) = 8$ và $h_3(H) = 13$ bằng chi phí đường tìm được.
- **Hiểu rõ hơn về cột updates và best_g:** Ở trace dòng 2 (S/L), nút có 3 con nhưng cột updates chỉ ghi 2 con là U và D. Con thứ ba quay về S có g = 2 lớn hơn best_g(S) = 0 nên bị bỏ (search.py dòng 131–133). Nhóm hiểu rõ hơn rằng generated đếm cả con quay về, còn updates và pushed chỉ đếm bản ghi cải thiện.
- **Hiểu rõ hơn về FIFO:** Ở lần 2 và 3, trace lấy S/L và S/R trước S/U và S/D vì f = 8 nhỏ hơn f = 10. FIFO chỉ phá hòa khi f bằng nhau: S/L (serial 3) được lấy trước S/R (serial 4) vì cùng f = 8 (search.py dòng 140).

04. KẾT QUẢ CHUNG — 36 CẤU HÌNH

Tệp CSV/JSON:
output/bat_buoc.csv và output/bat_buoc.json.

Lệnh chạy / order / model / số lần lặp:
python run.py --suite --tag bat_buoc / UDLR / unit, directional / 1.

Các ô N là số lần mở rộng; không phải số lần sinh con hay số byte RAM.

6. Mô hình unit

Case	Cost tối ưu (UCS)	N BFS	N A*/h1	N A*/h2	Giảm từ h1→h2
M1	8	310	18	12	33.33%
M2	14	2948	224	39	82.59%
M3	20	43513	3488	586	83.20%
M4	24	129053	16892	1047	93.80%

Công thức tỷ lệ giảm và một phép tính cụ thể:

- Tỷ lệ giảm = 1 - N/N0
- Tại M1, tỉ lệ giảm từ h1 -> h2 là = 1 – 12/18 = 33.33%

Cặp BFS/UCS và A* có cùng cost không? Bằng chứng:
Có. Trong mô hình unit, BFS, UCS và A* luôn có cùng cost. Bằng chứng là trong file bat_buoc.csv cột cost của các thuật toán luôn trùng nhau trong từng case. Với M1, cost đều = 8 qua các thuật toán BFS, UCS và A*. Tương tự với M2, M3, M4 có cost trùng khớp lần lượt là 14, 20, 24.

7. Mô hình directional

Case	BFS: bước / cost	UCS: bước / cost	N A*/h2	N A*/h3	Giảm từ h2→h3
M1	8/13	8/13	29	12	58.62%
M2	14/20	14/20	91	33	63.74%
M3	20/30	20/28	1173	403	65.64%
M4	24/38	24/36	6418	1057	83.53%

Một trường hợp BFS ít bước nhưng không ít chi phí:
Trường hợp M3: Cả BFS và UCS đều tìm ra lời giải có cùng độ dài là 20 bước. Tuy nhiên, BFS trả về tổng chi phí là cost = 30, trong khi UCS tối ưu hóa chi phí đường đi tốt hơn với cost = 28. Điều này xảy ra vì mô hình directional có trọng số chi phí khác nhau cho hướng di chuyển ngang và dọc (cost = 2 cho bước ngang và 1 cho bước dọc), và BFS chỉ xét theo số lượng bước đi ngắn nhất chứ không xét đến trọng số cạnh.

Một so sánh heuristic có kiểm soát:
Tại M2:
- Start = (1, 3, 8, 7, 2, 4, 5, 6, 0)
- Goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)
- Cost mode: directional - Order: UDLR.
- Kết quả: A* của h2 cần 91 lần mở rộng, trong khi A* của h3 chỉ cần 31 lần mở rộng.
- Lý do: Trong mô hình directional, các bước di chuyển ngang có chi phí lớn hơn (chi phí = 2). Việc điều chỉnh trọng số ở h_3 giúp hàm ước lượng bám sát chi phí thực tế hơn, khiến không gian tìm kiếm tập trung đi thẳng về đích và hạn chế tối đa việc mở rộng các nhánh thừa.

Nếu có đo thời gian: Không thực hiện.

05. DỮ LIỆU RIÊNG — 18 CẤU HÌNH

8. Hai trạng thái và đích mới

Tệp dữ liệu:
du_lieu_nhom.json

Goal chung cho N1/N2:
(1, 2, 3, 4, 5, 6, 0, 7, 8)

Nội dung	N1	N2
Start (9 số)	613502478	410532678
Cách tạo / seed / lịch sử	Tạo từ script `tao_du_lieu.py`/Seed: 2026/ Lịch sử: 16 bước ("U","R","U","L","D","R","R","U","L","D","L","U","R","R","D","L")	Tạo từ script `tao_du_lieu.py`/Seed: 2026/Lịch sử: 16 bước ("R","U","R","U","L","L","D","D","R","U","L","D","R","U","R","U")
Số bước xáo trộn	16	16
Số bước tối ưu do BFS	16	14
Kiểm khác mẫu / khả giải	Khác bàn cờ mẫu; Khả giải (có cùng tính chẵn lẻ số phép nghịch thế với goal)	Khác bàn cờ mẫu; Khả giải (có cùng tính chẵn lẻ số phép nghịch thế với goal)

Tại sao lịch sử xáo trộn không phải một chứng minh tối ưu:
Lịch sử xáo trộn chỉ là một chuỗi các bước di chuyển ngẫu nhiên hợp lệ đi từ trạng thái đích để tạo ra bàn cờ ban đầu. Chuỗi xáo trộn này có thể chứa các bước đi thừa, di chuyển lặp hoặc đi vòng. Do đó, độ dài của chuỗi xáo trộn (steps = 16) chỉ đại diện cho một đường đi khả thi chứ không đảm bảo là đường đi ngắn nhất.

Case / model	BFS: bước / cost	UCS: bước / cost	A*: h / bước / cost	Kiểm path
N1/unit	16/16	16/16	h1/16/16 h2/16/16	Khớp (16 bước)
N2/unit	14/14	14/14	h1/14/14 h2/14/14	Khớp (14 bước)
N1/directional	16/25	16/25	h1/16/25 h2/16/25 h3/16/25	Khớp (16 bước-cost 25)
N2/directional	14/22	14/22	h1/14/22 h2/14/22 h3/14/22	Khớp (14 bước - cost 22)

Tệp output/tutao.csv và JSON có đủ 18 dòng không?
Output/tutao.csv và JSON đủ 18 dòng.

Kết quả h(goal mới) và cách kiểm:
- h mới là 0.
- Cách kiểm: Trạng thái goal mới là (1, 2, 3, 4, 5, 6, 0, 7, 8). Khi đưa chính trạng thái này vào hàm đánh giá heuristic h, do tất cả các ô số đã nằm đúng vị trí chuẩn nên số ô sai vị trí h_1, tổng khoảng cách Manhattan h_2, và Weighted Manhattan h_3 đều trả về kết quả bằng 0.

9. Thử đổi thứ tự RDLU trên H

Dự đoán trước:
steps = 8, cost = 8, expanded = 12.

Kết quả / tệp trace riêng:
solved steps = 8 cost = 8 expanded = 12.

Điều thay đổi và điều không đổi; vì sao:

- Điều thay đổi: Thứ tự sinh nút con và trace khác nhau (do RDLU ưu tiên giải các bước R và D trước còn UDRL ưu tiên giải các bước U và D trước).
- Điều không đổi: steps, cost và expanded không thay đổi do mặc dù việc ưu tiên xét trạng thái khác nhau nhưng khi đưa vào biên A* sắp xếp lại theo giá trị của f thì vẫn tìm được kết quả tốt nhất khi xét f = h + g min.

STT	Tên test / dữ liệu	Kỳ vọng + cách suy ra	Kết quả thật	Người
1	test_trang _ thai _ dau_ trung _dich Dữ liệu: Vị trí bắt đầu đúng là G' (h=0)	- Nếu trạng thái bắt đầu trùng với Goal thì kiểm tra solve() với GOAL mới - Cost = 0 và steps = 0, path là [goal]	ok	Đoàn Mai Trúc Linh
2	test_chuyen_ dich_ mot_ buoc_ngang _doc Dữ liệu: vị trí ô trống chỉ cách G' đúng một nước đi	- Chi phí sau 1 bước đi trong mô hình directional và unit với mỗi bước đi dọc hoặc ngang để kiểm tra có trùng với đích G' hay không - Kiểm tra solve() khi quét ngang dọc với trường hợp unit (cost =1, steps =1) và directional (cost =2 với R,L và cost =1 với U,D; step = 1)	ok	Đoàn Mai Trúc Linh
3	test_dich_tuy_chinh Dữ liệu: - start = (6, 1, 3, 5, 0, 2, 4, 7, 8) - goal = (1, 2, 3, 4, 5, 6, 0, 7, 8) (start là bàn N1, goal là đích tùy chỉnh nhóm tự sinh trong du_lieu_nhom.json)	- Cần 16 bước, tương đương chi phí lời giải 16 trong mô hình unit (sử dụng heuristic là "manhattan"). - Chi phí là 25 trong mô hình directional, heuristic là "weighted" (16 bước, với 9 bước ngang * 2 + 7 bước dọc). - Kiểm tra solve() không dùng đầu vào/đích mặc định.	ok	Trần Thùy Dương
4	test_heuristic_bang_0_tai_ dich_va_bo_qua_o_trong Dữ liệu: - Đích tùy chỉnh sinh trong du_lieu_nhom.json: goal = (1, 2, 3, 4, 5, 6, 0, 7, 8) - Đích chuẩn GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)	- Heuristic tại goal = 0 với mọi h và mô hình. - Heuristic "zero" luôn bằng 0 ở mọi trạng thái, không cần model. * So sánh GOAL với đích tùy chỉnh goal, ta thấy 2 ô 7 và 8 của GOAL lệch so với đích tùy chỉnh 1 bước ngang mỗi ô. Như vậy: - "misplaced" trong "unit" cho h = 2 (tương ứng 2 ô sai vị trí). - "weighted" và "manhattan" trong unit cho h = 2 (tổng sai số theo Manhattan bằng 2). - "weighted" trong directional cho h = 4 (tổng sai số theo Manhattan bằng 2 theo phương ngang, cần nhân thêm trọng số 2). => Ô trống cũng dịch chuyển nhưng không được tính vào h.	ok	Trần Thùy Dương
5	test_vo_nghiem_do_parity Dữ liệu: Z = (1,2,3,4,5,6,8,7,0), đích GOAL = (1,2,3,4,5,6,7,8,0)	- Tính tay: bỏ 0, dãy của Z là 1 2 3 4 5 6 8 7 có đúng 1 nghịch thế (cặp 8,7), số lẻ; GOAL có 0 nghịch thế, số chẵn. Parity khác nhau và bất biến với bàn 3×3 nên Z vô nghiệm. - solve() trả status = "unsolvable" (không phải "limit"); expanded = generated = pushed = popped = 0, path = [], actions = [] vì parity được loại trước khi tìm kiếm. - Với max_expanded = 0 kết quả vẫn là "unsolvable", phân biệt với dừng do limit. - Đổi đích thành chính Z thì is_solvable(Z, Z) đúng và solve() trả solved, 0 bước, cost 0.	Ok	Hoàng Thị Ngọc Sơn
6	test_du_lieu_sai_va_hanh_dong_vuot_bien Dữ liệu: các trạng thái sai định dạng; corner = (0,1,2,3,4,5,6,7,8) (ô trống ở góc trên trái)	- validate_state() và solve() phải ném ValueError với: thiếu 1 phần tử, có số 9 thiếu số 8, trùng số 1, và chứa giá trị bool (False). - solve() ném ValueError khi order = "UDLL", model = "khong_ton_tai" hoặc algorithm = "dfs". - Tính tay: ô trống ở góc (0,0) chỉ đi được xuống (D) và sang phải (R), nên neighbors(corner) trả đúng ["D", "R"], không sinh U và L (vượt biên). - check_path() phải ném ValueError với đường đi dùng hành động U không hợp lệ, đường đi bị cắt bớt một trạng thái, và đường đi có đích cuối sai.	ok	Hoàng Thị Ngọc Sơn

STT	Tên test / dữ liệu	Kỳ vọng + cách suy ra	Kết quả thật	Người
7	test_duong_di_hop_le_va_dau_vao_bat_bien	Ô trống ở giữa, đi xuống rồi sang phải là tới đích. Model unit mỗi bước giá 1 nên cost = 2. check_path kiểm lại từng bước và phải trả đúng 2, khớp r['cost']. Hàm solve() tìm lời giải bằng A* với heuristic Manhattan và mô hình unit nhưng không làm thay đổi trạng thái đầu vào start. Phần tử đầu tiên của đường đi (r['path'][0]) phải trùng với trạng thái ban đầu.	Ok	Phạm Hồ Minh Thư
8	test_bfs_khong_toi_uu_gia_o_directional	Ở directional, U/D giá 1 còn L/R giá 2, nên một đường n bước có cost = n + (số bước L/R). BFS chỉ tối thiểu số bước và giữ đường nó gặp trước theo UDLR, không so chi phí. UCS luôn giữ đường rẻ nhất. Vậy BFS phải có đúng số bước ít nhất (20), UCS cho cost tối ưu (28), và cost BFS > cost UCS vì đường BFS gặp trước chứa nhiều bước ngang hơn.	ok	Phạm Hồ Minh Thư
9	test_doi_chieu_astar_zero_va_ucs Dữ liệu: Start = (1, 3, 6, 5, 0, 2, 4, 7, 8) Goal chuẩn	Thuật toán A* với h=0 cho ra kết quả trùng khớp với UCS trên mọi chỉ số: path, cost, expanded, generated, pushed, popped Hàm đánh giá của A* là f(s) = g(s) + h(s). Khi chọn h(s) = 0 với mọi trạng thái s, hàm đánh giá trở thành f(s) = g(s), đúng bằng chi phí thực tế đã đi. Lúc này, hàng đợi ưu tiên của A* và UCS mở rộng các nút theo thứ tự chi phí g(s) tăng dần và dùng chung quy tắc phá hòa FIFO, do đó toàn bộ quá trình tìm kiếm và các chỉ số đếm của hai thuật toán phải hoàn toàn giống nhau.	ok	Nguyễn Thanh Bình
10	test_doc_lap_trace_va_gioi_han_tai_nguyen Dữ liệu: Start = (1, 3, 6, 5, 0, 2, 4, 7, 8) Goal chuẩn trace_limit = 5 Giới hạn max_expanded=2	Bật trace_limit chỉ chứa tối đa 5 bản ghi log, không thay đổi các thông số tìm kiếm. Giới hạn max_expanded=2 khiến thuật toán dừng sớm; status trả về limit và path rỗng trace_limit chỉ đóng vai trò ghi nhận vết hoạt động vào cấu trúc dữ liệu phụ, không can thiệp vào logic chọn nút hay cập nhật best_g của A* max_expanded = 2 sẽ bị chạm ngay sau lượt mở rộng nút thứ 2. Thuật toán phải nhận diện đây là việc dừng do cạn kiệt tài nguyên.	ok	Nguyễn Thanh Bình

06. KIỂM THỬ DO NHÓM TỰ VIẾT

Tệp:
test_nhom.py. Log: output/ket_qua_test.txt.

Lệnh chạy / ngày chạy / tổng test / số đạt, lỗi:
Lệnh chạy: python -m unittest -v test_nhom

Ngày chạy: 23/10/2026

Tổng test: Bao gồm 10 test theo yêu cầu và 2 test bắt lỗi heuristic

Số đạt: 12

Tên ba test có kỳ vọng tính tay độc lập và trang minh chứng:

test_trang _ thai _ dau_trung _dich
test_chuyen_ dich_ mot_ buoc_ngang _doc
test_vo_nghiem_do_parity

(Trang minh chứng: ket_qua_test.txt)

Test bắt lỗi đếm cả 0:

- Tên: test_h_dem_ca_o_trong.
- Khẳng định: Heuristic đếm cả ô trống (số ô khác vị trí, kể cả ô 0) không chấp nhận được, vì tồn tại trạng thái mà h > h*. Mỗi nước đi làm lệch cả ô trống lẫn một ô số, nên heuristic này đếm dư 1 so với chi phí thực.
- Phản ví dụ: V = (1, 2, 3, 4, 5, 6, 7, 0, 8), G = (1, 2, 3, 4, 5, 6, 7, 8, 0). Chỉ cần 1 nước (đẩy ô trống sang phải) nên h*(V) = 1, nhưng hàm sai cho h(V) = 2 (ô 8 và ô 0 đều lệch) > h*. Vậy h(V) > h*(V), vi phạm tính chấp nhận được.

Test bắt lỗi goal hard-code:

- Tên: test_h_dung_vi_tri_dich_co_dinh.
- Khẳng định: Heuristic dùng vị trí đích cố định theo GOAL chuẩn, bỏ qua goal tùy chỉnh, không hợp lệ với goal bất kỳ, vì nó vi phạm h(G') = 0 tại chính đích G'.
- Phản ví dụ:
- G' = (1, 2, 3, 4, 5, 6, 0, 7, 8). Tại đích G' thì h*(G') = 0, nhưng hàm sai vẫn tính Manhattan theo GOAL chuẩn: ô 7 ở vị trí (2, 1) thay vì (2, 0) lệch 1, ô 8 ở vị trí (2, 2) thay vì (2, 1) lệch 1, nên h(G') = 2 ≠ 0.

Test kiểm path và tính lại cost:

test_duong_di_hop_le_va_dau_vao_bat_bien

Test xác nhận không sửa đầu vào / trace không đổi kết quả:

test_duong_di_hop_le_va_dau_vao_bat_bien

Test chưa đạt hoặc giới hạn chưa kiểm:

Đã kiểm thử mọi giới hạn đề bài yêu cầu, mọi test đều đã thông qua.

07. NHẬN XÉT, NGUỒN VÀ TÁI TẠO

10. Ba kết luận chính có minh chứng

Kết luận 1 — heuristic và mở rộng nút:

Ở mô hình unit, A* với h_2 ("manhattan") mở rộng ít nút hơn h_1 ("misplaced") mà vẫn tối ưu: bàn M3 là 586 so với 3488 nút (giảm 83,2%, bat_buoc.csv dòng 22–23), bàn M4 là 1047 so với 16892 (giảm 93,8%, dòng 31–32), cost giữ nguyên 20 và 24. Ở directional, h_3 (weighted) giảm tiếp so với h_2: M4 còn 1057 so với 6418 nút (giảm 83,5%, dòng 36–37).

Kết luận 2 — số bước và chi phí:

Ở directional, BFS tối thiểu số bước nhưng không tối thiểu chi phí. Bàn M3: BFS và UCS cùng 20 bước, nhưng BFS cost 30 còn UCS cost 28 (dòng 24–25). Bàn M4: cùng 24 bước, cost 38 so với 36 (dòng 33–34). Nguyên nhân là L/R giá 2, nên đường cùng số bước nhưng nhiều bước ngang sẽ đắt hơn, và BFS lấy đường gặp đầu tiên theo UDLR thay vì đường rẻ nhất. Ở M1 và M2 hai thuật toán trùng cost (13 và 20), nên chênh lệch chỉ xuất hiện khi có nhiều đường cùng số bước.

Kết luận 3 — một lỗi đã sửa hoặc điểm hiểu rõ hơn:

Test ban đầu dùng bàn hai bước (1, 2, 3, 4, 0, 6, 7, 5, 8). Trên bàn đó BFS và UCS đều có cost 3 (đường D,R duy nhất), nên test không chứng minh được BFS không tối ưu. Nhóm đổi sang M3, nơi BFS cost 30 còn UCS cost 28 với cùng 20 bước, rồi dùng assertGreater để khẳng định.

11. Giới hạn và nguồn dùng lại

Giới hạn 1:

Mọi so sánh heuristic chỉ được thực hiện trên bốn bàn M1–M4 và hai bàn N1–N2, với thứ tự sinh trạng thái con UDLR. Trong các trường hợp đã thử, A* sử dụng h_2 mở rộng ít nút hơn A* sử dụng h_1 (ví dụ, trên M4 với mô hình chi phí unit, số nút mở rộng lần lượt là 1.047 và 16.892). Tương tự, trên M4 với mô hình directional, h_3 mở rộng 1.057 nút, ít hơn h_2 với 6.418 nút. Tuy nhiên, đây chỉ là kết quả quan sát trên tập kiểm thử đã chọn, chưa đủ để khẳng định h_2 luôn hiệu quả hơn h_1 hoặc h_3 luôn hiệu quả hơn h_2 trên mọi bàn. Do số lượng bàn kiểm thử còn hạn chế, kết quả chưa đại diện cho toàn bộ không gian trạng thái của bài toán 8-puzzle.

Giới hạn 2:

Trong mô hình chi phí directional, BFS tìm được đường đi có chi phí 30 trên M3 và 38 trên M4, trong khi UCS tìm được đường đi có chi phí tương ứng là 28 và 36. Điều này cho thấy BFS không đảm bảo tìm được đường đi có chi phí thấp nhất trong mô hình chi phí không đồng nhất. Tuy nhiên, các kết quả cụ thể còn phụ thuộc vào thứ tự sinh trạng thái con UDLR và cách xử lý các trạng thái trùng lặp. Thay đổi thứ tự này có thể khiến BFS tìm được một đường đi khác có cùng số bước nhưng chi phí khác.

Nguồn thuật toán/mã/công cụ hỗ trợ đã dùng, phần giữ nguyên và cách kiểm chứng:

Nhóm sử dụng và giữ nguyên bộ mã nguồn cốt lõi từ (gồm các tệp search.py, puzzle.py, run.py, tao_du_lieu.py,...) bao gồm các thuật toán tìm kiếm (bfs, ucs, astar), kết hợp với trợ lý AI Gemini để hỗ trợ rà soát logic kiểm thử và tối ưu hóa cấu trúc tệp test_nhom.py. Để kiểm chứng tính đúng đắn của phần giữ nguyên, dùng test_nhom.py nhằm xác thực tính bất biến , đảm bảo các hàm tìm kiếm không làm thay đổi hoặc làm sai lệch dữ liệu đầu vào sau khi thực thi, đồng thời kiểm tra trạng thái đầu tiên trong chuỗi kết quả trả về (path[0]) luôn khớp tuyệt đối với trạng thái khởi đầu ban đầu để bảo toàn toàn vẹn luồng dữ liệu.

12. Tái chạy và chuẩn bị bảo vệ

Kiểm tra trước nộp	Kết quả / tệp / người kiểm
Đã có báo cáo, PDF tính tay, 6–8 slide	Đã hoàn thành/ TinhTay.pdf, ThuyetTrinh.pdf, BaoCao.pdf/ Đoàn Mai Trúc Linh
36+18 cấu hình và JSON chứa đường	Đã hoàn thành/Xuất 4 file .csv và .json, bat_*buoc.csv,* *bat*_buoc.json, tu_*tao.csv, tu*_tao.json / Trần Thùy Dương
Ít nhất 10 test riêng, log thật	Đã hoàn thành/ test_nhom.py và ket_qua_test.txt/ Đoàn Mai Trúc Linh

Kiểm tra trước nộp	Kết quả / tệp / người kiểm
README, dữ liệu, Python và đóng góp	Đã hoàn thành tệp READMENHOM1.md, tạo dữ liệu mới khác Goal và N1 N2, sử dụng nguyên phần file mẫu .py, hoàn thành báo cáo và các file kết quả / README_NHOM1.md, du_lieu_nhom.json, test_nhom.py, BaoCao.pdf, ThuyetTrinh.pdf, TinhTay.pdf / Cả nhóm
Giải nén chạy lại trên máy thành viên khác	Đã hoàn thành/ Nhom1_8Puzzle_GiuaKy.zip/ Đoàn Mai Trúc Linh
Cả năm người tập trả lời 15–20 phút/nhóm	Đã hoàn thành/ cả nhóm

Tên ZIP nộp:
Nhom1_8Puzzle_GiuaKy.zip

Thành viên xác nhận / ngày:

Đoàn Mai Trúc Linh
Trần Thùy Dương
Hoàng Thị Ngọc Sơn
Phạm Hồ Minh Thư
Nguyễn Thanh Bình

24/10/2026.